

CS355 Lab 1 — Batch A (14:00–15:25)

Language: Scheme (DrRacket, `#lang sicp`)

Q1 (1 mark) — `two-larger-squares` (file name: q1.scm)

Write a procedure `two-larger-squares` that takes three numbers `a` `b` `c` and returns the sum of the squares of the **two larger** of the three. You do not need recursion for this — `cond/if` and arithmetic are enough.

```
(two-larger-squares 1 2 3) => 13      ; 2^2 + 3^2
(two-larger-squares 5 2 3) => 34      ; 5^2 + 3^2
(two-larger-squares 3 3 3) => 18
```

Q2 (1 mark) — `sum-of-digits` (file name: q2.scm)

Write a procedure `sum-of-digits` that takes a non-negative integer `n` and returns the sum of its decimal digits. Write this as an **iterative process** — i.e., use a tail-recursive helper procedure with an accumulator. (`quotient` and `remainder` will be useful.)

```
(sum-of-digits 123) => 6
(sum-of-digits 7)  => 7
(sum-of-digits 0)  => 0
```

Q3 (1 mark) — `sum-terms` (file name: q3.scm)

Write a procedure `sum-terms` that takes four arguments — a procedure `term`, a lower bound `a`, a procedure `next`, and an upper bound `b` — and returns:

```
(term a) + (term (next a)) + (term (next (next a))) + ... + (term b)
```

i.e. it applies `term` to every value from `a` to `b` (stepping via `next`) and adds up the results. This is a **higher-order function**: `term` and `next` are procedures you're given, not ones you write yourself for this part.

```
(sum-terms (lambda (x) x) 1 (lambda (x) (+ x 1)) 5)      => 15 ;  
1+2+3+4+5  
(sum-terms (lambda (x) (* x x)) 1 (lambda (x) (+ x 1)) 3) => 14 ;  
1^2+2^2+3^2
```

Take-home practice (ungraded)

total-led-blocks

Techfest is building a stepped LED display for the main stage. The display has steps numbered **a** through **b**, and step **i** needs **i²** LED blocks (step 1 needs 1 block, step 2 needs 4, step 3 needs 9, and so on — it gets wider as you go up). Use your **sum-terms** from Q3 (not a hand-written loop) to write **total-led-blocks**, which takes **a** and **b** and returns the total number of LED blocks needed for steps **a** through **b**.

```
(total-led-blocks 1 3) => 14 ; 1^2 + 2^2 + 3^2  
(total-led-blocks 5 5) => 25
```

Once you're done, feel free to also try Batch B's Q1–Q3 for extra practice — same topics, different problems.